Machine Learning for Customer Churn Prediction

Nelson David Posso Suarez
Javier Alejandro Penagos Hernandez
Nicolas Castro Rivera

1 Executive Summary

Unsupervised learning provides techiniques useful for different problems that complement the projects. The re-creation of existing data, the anomaly detection or generate new data similar to existing data. Can be help in the domain of Churn Prediction, in special for working with a dataset like the KKBox selected for this project.

2 Motivation

The contribution of this workshop to the project, is implement unsupervised learning for strategy parts in the Customer Churn Prediction system. There is a problem with the unbalanced dataset, then the experiment will consist in explore the AE and VAE for show how good reply existing customers and then implement for generate customer churns.
Purpose of this new customers generated with real customer churns, is generate more data and get a better performace, that the models of w1 got. The experiment will conclude if this whether or not this is feisible depends on the results.

3 AE and VAE

Autoencoder and variational autoencoder are unsupervised neural network architectures that learn compressed representations of data without using labels. Both were designed first at the architectural level — defining inputs, hidden layers, bottleneck, and loss function — and then implemented in PyTorch (`torch.nn.Module`).

Standard Autoencoder (AE). The AE was designed as a deterministic encoder–decoder for tabular data: the encoder compresses the input to a fixed-size code vector and the decoder reconstructs the original features from it. Once the design was settled, it was implemented across two modules:

- `autoencoder_ae.py` (`workshops/w4/src/autoencoder/`): defines `AEConfig` (frozen dataclass with all hyperparameters), `Encoder` (stacked linear layers with ReLU ending in a linear projection to z), `Decoder` (mirrored layers branching into a numeric linear head and per-category logit

heads), `Autoencoder` (combines both modules with `encode` / `decode` / `forward` methods), and `ReconstructionLoss` (MSE for numeric features plus mean cross-entropy for categorical features).

- `train_ae.py` (`workshops/w4/src/train/`): defines `AETrainer` (training loop with Adam optimiser, per-epoch logging, and best-state tracking), `EarlyStopping` (patience-based stopping on validation loss), `build_loader` (wraps tensors in a `DataLoader`), and checkpoint utilities `save_checkpoint` / `load_checkpoint` that persist the model, config, preprocessor, and training history.

Variational Autoencoder (VAE). The VAE extends the AE with a probabilistic bottleneck: the encoder outputs a mean $\boldsymbol{\mu}$ and a log-variance $\log \boldsymbol{\sigma}^2$ instead of a deterministic code, and a sample $\mathbf{z}$ is drawn via the reparameterization trick during training. The decoder is shared with the AE. A KL divergence term is added to the loss, weighted by β and linearly annealed during the first epochs to avoid posterior collapse. The implementation is split into two modules:

- `autoencoder_vae.py` (`workshops/w4/src/autoencoder/`): defines `VAEConfig` (extends `AEConfig` with `beta`, `kl_anneal_epochs`, and `logvar_clamp`), `VAEEncoder` (shared trunk ending in parallel `mu_head` and `logvar_head`), `VAE` (reparameterization, `sample` method for generation, reuses `Decoder` from the AE module), and `ELBOLoss` (reconstruction summed over feature dimensions plus β-weighted KL divergence).

- `train_vae.py` (`workshops/w4/src/train/`): defines `VAETrainer` (training loop that advances the KL annealer each epoch and tracks active latent units to detect posterior collapse) and `KLAnnealer` (linear warm-up schedule for β). Reuses `EarlyStopping` and `build_loader` from `train_ae.py`; early stopping is delayed until after the KL warm-up phase.

3.1 Data and preprocessing

The autoencoders reuse the tabular export built in Workshop 1 (version 5), where the *members*, *transactions* and *user_logs* tables are condensed into a single record of 59 features per customer (56 numerical and 3 categorical). The February cohort (992,931 customers, 6.39% churn) is used for development and the March cohort (970,960 customers, 8.99% churn) as the held-out test set; 881,701 customers appear in both months.

The Workshop 1 split (stratified and hash-based on `msno`) is reused so that the same customers are used across workshops. The churn label is never fed to the networks; it is only used to select the training population. The AE and the VAE are trained on the 697,095 non-churn customers of the *fit* partition and validated on the 116,183 non-churn customers of the *stop* partition, while the full March cohort, including churners, is used for testing.

A preprocessor is fitted on the training population only. Numerical features are imputed with the median, skewed features are transformed with a signed $\log(1+|x|)$, clipped to the 0.1% and 99.9% quantiles and standardized; one constant column was removed, leaving 55 numerical inputs. The categorical features (`city`, `registered_via`, `last_payment_method_id`) are mapped to vocabularies of 22, 6 and 34 levels and one-hot encoded inside the network, giving an input dimension of 117. An inverse transform

maps reconstructions back to the original 59 columns. The AE and the VAE share the same fitted preprocessor.

3.2 AE Architecture

The autoencoder receives tabular input of dimension $d_{\text{in}} = 117$: 55 standardised numerical features concatenated with the one-hot encodings of the three categorical columns (`city`: 22, `registered_via`: 6, `last_payment_method_id`: 34).

Encoder. Two fully-connected layers with decreasing hidden units compress the input to the bottleneck:

$$\mathbb{R}^{117} \xrightarrow{W_1,\,\text{ReLU}} \mathbb{R}^{64} \xrightarrow{W_2,\,\text{ReLU}} \mathbb{R}^{32} \xrightarrow{W_z} \mathbb{R}^{z}.$$

Bottleneck. Three latent dimensions were evaluated: $z \in \{8, 16, 32\}$. A linear projection (no activation) maps the last hidden representation to the code vector $\mathbf{z}$, which is the only information passed to the decoder.

Decoder. The decoder mirrors the encoder with reversed hidden widths $[32, 64]$ and then branches into two output heads:

- *Numeric head*: a linear layer (no activation) reconstructing the 55 standardised numerical features.
- *Categorical heads*: one independent linear layer per categorical feature, producing un-normalised class logits (22, 6, and 34 dimensions respectively).

Loss function. The reconstruction loss combines mean squared error for the numerical part with the mean cross-entropy over all categorical heads, controlled by a scalar weight $\lambda = 1.0$:

$$\mathcal{L} = \underbrace{\text{MSE}(\hat{\mathbf{x}}_{\text{num}},\, \mathbf{x}_{\text{num}})}_{\text{numeric}} + \lambda\, \underbrace{\frac{1}{C}\sum_{c=1}^{C} \text{CE}(\hat{\mathbf{x}}_c,\, x_c)}_{\text{categorical}}.$$

Table 1 lists the hyperparameters used and the rationale behind each choice.

Table 1: AE hyperparameters and selection rationale.

Hyperparameter	Value	Rationale
`hidden_dims`	(64, 32)	Funnel shape: halves width at each layer from 117 to 64, 32, then z. Two layers suffice for tabular inputs of this size; a common heuristic.
`z_dim`	16	Chosen by experiment (sweep over 8 / 16 / 32): balances reconstruction quality and compression.
`learning_rate`	10^{-3}	Default recommended for Adam [1]. Loss curves decrease smoothly without oscillation, confirming it is adequate.
`batch_size`	512	Gives ≈1,362 updates per epoch (697,095 / 512); consistent with the batch size used in Workshop 2.
`max_epochs`	50	Upper bound set by the workshop guidelines (30–50 epochs).
`patience`	5	Early stopping: training halts if validation loss does not improve for 5 consecutive epochs.
`dropout`	0.0	No dropout needed; train and validation curves track closely, indicating no overfitting.
`weight_decay`	0.0	Same reasoning as dropout; regularisation is not required given the data volume.
`cat_loss_weight`	1.0	Numerical and categorical components contribute equally to the total loss.
`seed`	110	Same seed used in Workshops 1 and 2 for reproducibility across experiments.

Figure 1 illustrates the full encoder–bottleneck–decoder pipeline.

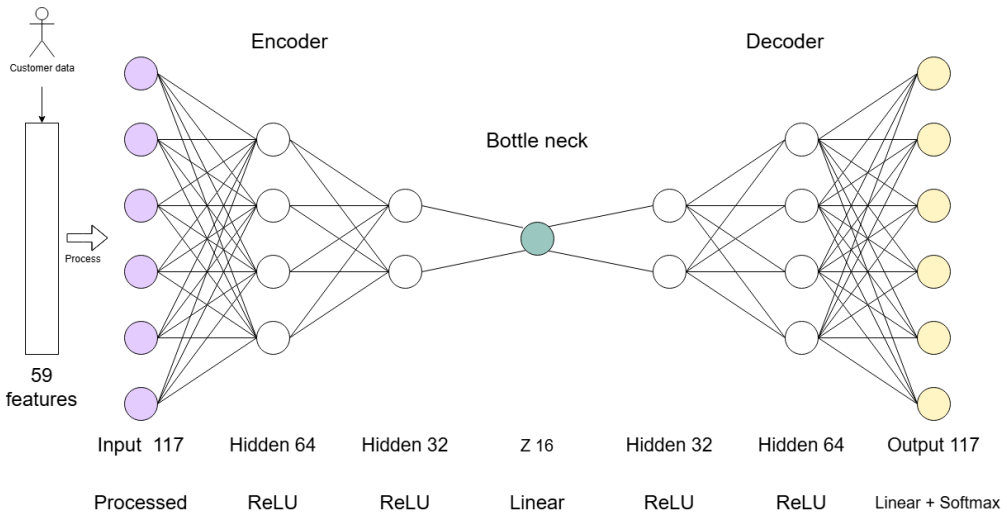


Figure 1: Standard Autoencoder architecture. The encoder compresses the 117-dimensional input to a z-dimensional code; the decoder reconstructs numerical features via a linear head and categorical features via per-class logit heads.

3.3 AE Train

All three latent configurations share the same training protocol. The Adam optimiser [1] is used with learning rate 10^{-3}, no weight decay, and a batch size of 512. Training runs for up to 50 epochs with early stopping (patience 5) monitoring the validation reconstruction loss. The model state corresponding to the lowest validation loss is restored at the end of training.

The training set consists of 697,095 non-churn customers from the February fit partition; the 116,183 non-churn customers of the stop partition are used as the validation set. The churn label is never seen during training.

Table 2 summarises the final train and validation losses for each bottleneck size. None of the three runs triggered early stopping; all completed the full 50 epochs.

Table 2: AE training results by latent dimension (epoch 50).

z	Train loss	Val loss	Train MSE	Val MSE
8	0.1274	0.1274	0.1250	0.1256
16	0.0443	0.0427	0.0433	0.0421
32	0.0161	0.0157	0.0156	0.0152

Reconstruction error drops sharply as z grows: z=32 achieves a validation loss roughly 8× lower than z=8, confirming that larger bottlenecks retain more customer-level information. Figure 2 shows the training and validation loss curves for all three runs.

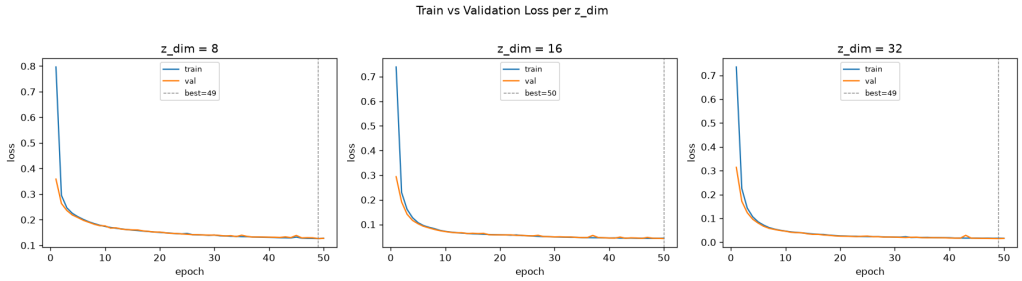


Figure 2: Training and validation reconstruction loss per epoch for $z \in \{8, 16, 32\}$. All curves converge smoothly without overfitting, and larger bottlenecks achieve consistently lower loss.

3.4 VAE Architecture

The VAE shares the same input representation as the AE (117 dimensions: 55 numerical and 62 one-hot). The key difference lies in the encoder and the loss function.

Encoder. A shared trunk identical to the AE encoder processes the input through two hidden layers:

$$\mathbb{R}^{117} \xrightarrow{W_1,\,\mathrm{ReLU}} \mathbb{R}^{64} \xrightarrow{W_2,\,\mathrm{ReLU}} \mathbb{R}^{32}.$$

Two parallel linear heads then project the trunk output to the parameters of the posterior distribution:

$$\boldsymbol{\mu} = W_\mu\,\mathbf{h}, \qquad \log \boldsymbol{\sigma}^2 = \mathrm{clamp}(W_\sigma\,\mathbf{h},\, -10,\, 10).$$

Bottleneck. A sample $\mathbf{z} \in \mathbb{R}^z$ is drawn during training via the reparameterization trick:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

At inference the deterministic mean $\boldsymbol{\mu}$ is used instead. The same three latent dimensions as the AE were evaluated: $z \in \{8, 16, 32\}$; z=16 was selected.

Decoder. The same `Decoder` module from the AE is reused unchanged: mirrored hidden layers $[32, 64]$ with ReLU, a linear numeric head, and per-category logit heads.

Loss (ELBO). The training objective is the Evidence Lower Bound, composed of a reconstruction term and a KL divergence weighted by β:

$$\mathcal{L}_{\mathrm{ELBO}} = \underbrace{\sum_{\text{num}} \mathrm{MSE}_i + \sum_{c=1}^{C} \mathrm{CE}_c}_{\text{reconstruction}} + \beta \underbrace{\left(-\tfrac{1}{2} \sum_{j=1}^{z} \big(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\big) \right)}_{\text{KL divergence}}.$$

β is linearly annealed from 0 to its target value over the first 10 epochs to prevent posterior collapse. Three values were evaluated: $\beta \in \{0.5, 1.0, 4.0\}$.

Table 3 lists the VAE-specific hyperparameters and their rationale.

Table 3: VAE-specific hyperparameters and selection rationale. All other hyperparameters match Table 1.

Hyperparameter	**Value**	**Rationale**
`beta`	1.0	Chosen by experiment (sweep over 0.5 / 1.0 / 4.0): balances reconstruction quality and latent space regularity. Standard β-VAE baseline.
`kl_anneal_epochs`	10	Linear warm-up prevents the KL term from dominating in early epochs, which would collapse the posterior before the decoder learns to reconstruct.
`logvar_clamp`	$(-10, 10)$	Clamps $\log \sigma^2$ to avoid numerical overflow in $\exp(\log \sigma^2)$ during the reparameterization step.

Figure 3 illustrates the full VAE pipeline.

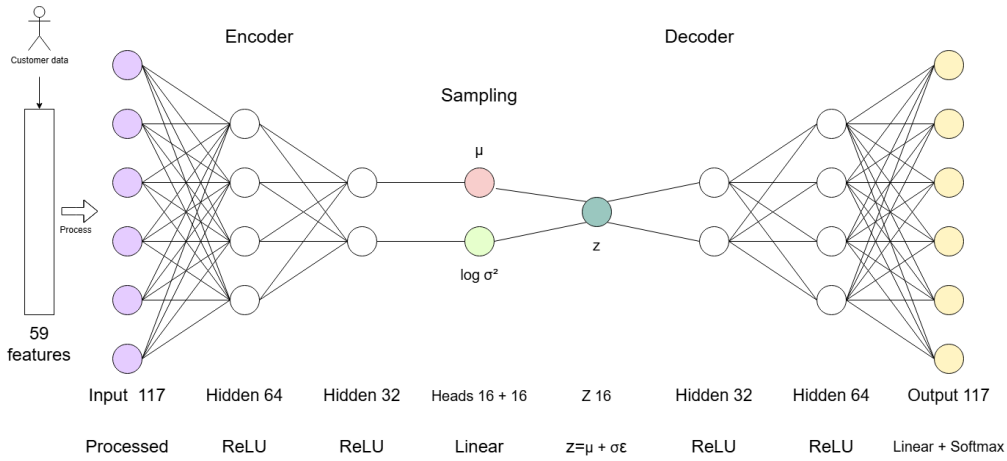


Figure 3: Variational Autoencoder architecture. The encoder trunk branches into $\boldsymbol{\mu}$ and $\log \boldsymbol{\sigma}^2$ heads; a sample is drawn via the reparameterization trick and passed to the shared decoder.

3.5 VAE Train

The VAE follows the same training protocol as the AE (Adam, lr $= 10^{-3}$, batch size 512, up to 50 epochs), with two differences: early stopping is delayed until after the KL warm-up phase (epoch > 10), and the KL annealer advances β_t linearly each epoch. Three values of β were evaluated: $\{0.5, 1.0, 4.0\}$.

Unlike the AE runs, all three VAE experiments triggered early stopping before reaching epoch 50. The active latent units metric — the number of dimensions with $\mathrm{Var}(\boldsymbol{\mu}) > 0.01$ across the validation set — is tracked each epoch to monitor posterior collapse.

Table 4 summarises the results at the best checkpoint for each run.

Table 4: VAE training results by β at the best validation epoch.

β	Stop ep.	Best ep.	Val ELBO	Val recon	Val KL	Active units
0.5	26	21	10.575	3.603	13.943	15
1.0	27	22	15.304	5.997	9.308	13
4.0	26	21	28.958	16.355	3.151	6

Higher β penalises the KL term more strongly, forcing the posterior closer to $\mathcal{N}(\mathbf{0}, \mathbf{I})$ at the cost of reconstruction quality: β=4.0 achieves the lowest KL (3.15) but the worst reconstruction (16.36) and collapses 10 of the 16 latent dimensions. β=0.5 retains 15 active units and the best reconstruction (3.60) but with a less regular latent space. β=1.0 was selected as the best trade-off. Figure 4 shows the training and validation ELBO curves for all three runs.

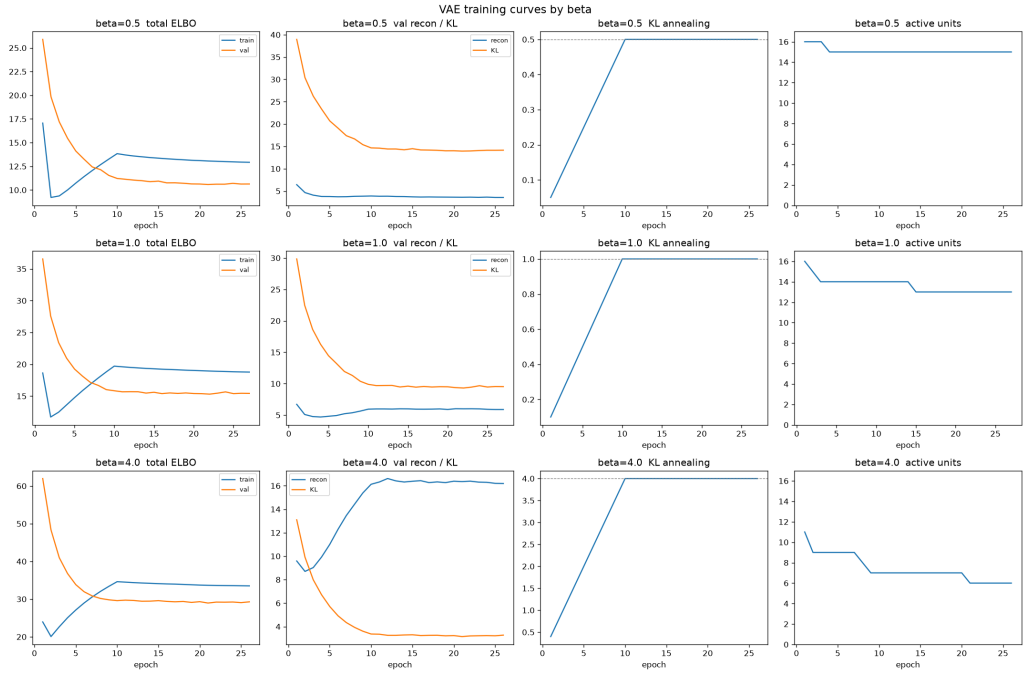


Figure 4: Training and validation ELBO per epoch for $\beta \in \{0.5, 1.0, 4.0\}$. The KL warm-up phase (first 10 epochs) is visible as a rising loss before convergence. All runs triggered early stopping before epoch 50.

4 Reconstruction quality

5 Anomaly detection

6 Latent space visualization

7 Analysis

References

[1] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.

[2] Rizvi, A. (2018). *WSDM – KKBox's Churn Prediction Challenge*. Medium. Retrieved from `https://asad-99rizvi.medium.com/wsdm-kkboxs-churn-prediction-challenge-d7c9cfa21cbd`